

Лабораторная работа №29. REST API: контроллеры и маршруты

Цель работы:

- Понять принципы архитектуры REST и что делает API «RESTful»;
- Создать первый контроллер с полным набором CRUD-маршрутов;
- Освоить разницу между Minimal API и Controller-based API;
- Научиться тестировать API через Swagger UI и расширение REST Client в VS Code;
- Освоить правильные HTTP-статусы и формат ответов.

Краткая теория (вводная часть)

Что такое REST?

REST (Representational State Transfer) — это архитектурный стиль для построения веб-API. REST — не протокол и не стандарт, а набор принципов, которым должен следовать API, чтобы называться **RESTful**.

Основные принципы REST:

Принцип	Описание
Клиент-сервер	Клиент и сервер независимы друг от друга. Клиент не знает, как хранятся данные на сервере
Stateless	Каждый запрос самостоятелен. Сервер не хранит состояние между запросами. Аналог: банкомат — он не помнит, что вы уже вставляли карту минуту назад.
Единый интерфейс	Ресурсы идентифицируются через URL, операции — через HTTP-методы. Вместо <code>getTasks()</code> , <code>deleteTask()</code> , <code>addTask()</code> (как в обычном коде) REST использует один URL (<code>/tasks</code>) и HTTP-методы (GET, POST, DELETE). Адрес — что, метод — как.
Кэшируемость	Ответы могут быть закэшированы клиентом
Слоистость	Клиент не знает, обращается ли он напрямую к серверу или через прокси

Ресурсы и HTTP-методы

В **REST** любые данные — это **ресурсы**. Каждый ресурс имеет уникальный адрес (URL), а операции над ним определяются HTTP-методом.

Например, ресурс **tasks** (задачи):

Метод	URL	Действие	Аналог в С#
GET	/api/tasks	Получить все задачи	GetAll()
GET	/api/tasks/5	Получить задачу с id=5	GetById(5)
POST	/api/tasks	Создать новую задачу	Create(task)
PUT	/api/tasks/5	Обновить задачу с id=5	Update(5, task)
DELETE	/api/tasks/5	Удалить задачу с id=5	Delete(5)

Сравнение с JavaScript (fetch): В лабораторных по js вы уже делали запросы через **fetch**. Теперь вы будете создавать серверную сторону — именно тот API, к которому мог бы обращаться ваш JavaScript-код.

Minimal API vs Controller-based API

В ASP.NET Core есть два способа создавать маршруты:

	Minimal API	Controller-based API
Синтаксис	app.MapGet(...) прямо в Program.cs	Отдельные классы-контроллеры
Когда использовать	Маленькие проекты, микросервисы	Средние и крупные проекты
Структура	Всё в одном файле	Логика разбита по файлам
Что изучали	Предыдущие лабораторные	Эта лабораторная работа

В реальных проектах используют **контроллеры** — это позволяет держать код организованным, когда маршрутов становится много.

HTTP-статусы ответа

Правильно возвращать не только данные, но и нужный статус:

Код	Название	Когда использовать
200 OK	Успех	Данные успешно получены/обновлены
201 Created	Создано	Ресурс успешно создан (ответ на POST)
204 No Content	Нет содержимого	Успешно, но возвращать нечего (ответ на DELETE)
400 Bad Request	Плохой запрос	Неверный формат данных от клиента
404 Not Found	Не найдено	Ресурс с таким id не существует
500 Internal Server Error	Ошибка сервера	Что-то сломалось на сервере

Почему это важно?

Когда ваш **JavaScript** делает **fetch**, он смотрит на статус ответа (**response.ok, response.status**). Если сервер всегда возвращает 200, клиент не поймёт, произошла ошибка или нет.

Шаг 1. Подготовка проекта

1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab29_RestAPI
```

```
cd Lab29_RestAPI
```

```
mkdir img
```

1.2. Создание проекта Web API

1. Используем шаблон **webapi** — он создаёт проект с поддержкой контроллеров:

```
dotnet new webapi -n TaskApi
```

2. Откройте проект в VS Code:

```
code .
```

3. Откройте терминал и выполните:

```
cd TaskApi
```

```
dotnet restore
```

1.3. Первый запуск и знакомство со Swagger

В **.NET 10** убрали **Swagger** из шаблона и из зависимостей. Раньше пакет подключался автоматически. Теперь его вообще нет в проекте.

1. Установим его, введите в терминале:

```
dotnet add package Swashbuckle.AspNetCore
```

```
dotnet restore
```

Обратите внимание, у вас должен быть установлен .NET 10, и в настройках csproj версия должна быть 10! Иначе будут ошибки при добавлении!!!

2. Обновите файл **Program.cs**

```

var builder = WebApplication.CreateBuilder(args);

// Регистрация сервисов
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

Строка	Что делает
AddControllers()	Регистрирует поддержку контроллеров
AddEndpointsApiExplorer()	Позволяет Swagger анализировать маршруты
AddSwaggerGen()	Регистрирует генератор документации Swagger
UseSwagger()	Добавляет middleware для отдачи спецификации OpenAPI
UseSwaggerUI()	Добавляет middleware для веб-интерфейса Swagger
app.Environment.IsDevelopment()	Swagger включается только в режиме разработки
UseHttpsRedirection()	Перенаправляет HTTP → HTTPS
UseAuthorization()	Middleware авторизации (пока не используем)
MapControllers()	Регистрирует все маршруты из всех контроллеров

3. Запустите проект:

dotnet run

В терминале появится URL:

```

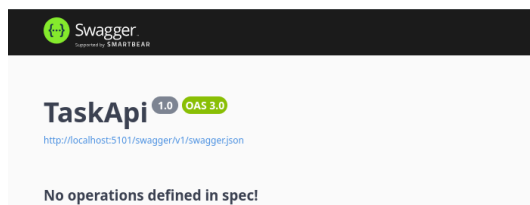
Используются параметры запуска из /home/denis/Lab
Сборка...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5101
info: Microsoft.Hosting.Lifetime[0]

```

Откройте браузер и добавьте к нему **/swagger:**

http://localhost:XXXX/swagger

Вы увидите **Swagger UI** — автоматически сгенерированную документацию вашего API с возможностью тестирования прямо в браузере:



Что такое Swagger (OpenAPI)?

Swagger — это инструмент, который читает ваш код и автоматически создаёт интерактивную документацию. Остановите сервер (**Ctrl+C**).

1.4. Инициализация Git-репозитория

1. Выполните в терминале:

```
cd ..
```

```
git init -b main
```

```
dotnet new gitignore
```

```
touch .editorconfig
```

2. Добавьте в ваш **.editorconfig** файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

3. Далее в терминале:

```
touch README.md
```

```
git add .
```

```
git commit -m "Initial commit: Web API project created"
```

4. Создайте пустой **public-репозиторий** на GitHub с именем: **Lab29_RestAPI**

Не добавляйте README, .gitignore, License

5. Привяжите и отправьте:

```
git remote add origin <URL_репозитория>
```

```
git push -u origin main
```

6. Сделайте **скриншот терминала** с выполненными командами (commit, remote add, push) и сохраните в папку **img** с именем: **gitPushLab29_FIO.png**

Выполните в терминале:

```
git add .
```

git commit -m "Added push screenshot"

git push

Шаг 2. Создание первой модели

2.1. Создание модели Task

В реальных приложениях данные описываются через **модели** — классы C#. Создадим папку для моделей и первую модель.

1. Создайте папку **Models** внутри **TaskApi**
2. Создайте файл **Models/TaskItem.cs**
3. Откройте файл и вручную перепишите следующий код:

```
namespace TaskApi.Models;

public class TaskItem {
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public bool IsCompleted { get; set; } = false;
    public DateTime CreatedAt { get; set; } = DateTime.Now;
    public string Priority { get; set; } = "Normal";
}
```

Разберём каждое свойство:

Свойство	Тип	Описание
Id	int	Уникальный идентификатор задачи
Title	string	Заголовок задачи (обязательное поле)
Description	string	Подробное описание задачи
IsCompleted	bool	Выполнена ли задача
CreatedAt	DateTime	Дата и время создания
Priority	string	Приоритет: Low / Normal / High

Сравнение с JavaScript: В JavaScript вы описывали объект как { id: 1, title: "...", isCompleted: false }. В C# это тот же объект, но с явными типами для каждого поля — компилятор проверит корректность данных ещё до запуска.

2.2. Создание DTO (Data Transfer Object)

В реальном API часто создают отдельный класс для **входящих** данных — это называется **DTO**. Например, при создании задачи клиент не должен передавать **Id** (он генерируется сервером) и **CreatedAt** (устанавливается автоматически).

Клиент отправляет:

CreateTaskDto

Сервер создаёт:

TaskItem

Title	→	Title
Description	→	Description
Priority	→	Priority
		Id ← генерирует сервер
		CreatedAt ← генерирует сервер
		IsCompleted ← всегда false при создании

1. Создайте файл **Models/CreateTaskDto.cs** и перепишите в него код:

```
namespace TaskApi.Models;

public class CreateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Priority { get; set; } = "Normal";
}
```

Зачем нужны DTO? Представьте, что клиент намеренно передаёт **Id** = 9999 или **CreatedAt** = 1970-01-01. Используя DTO, мы принимаем только те поля, которые разрешаем изменять — всё остальное устанавливает сервер.

2. Также создайте **Models/UpdateTaskDto.cs** и перепишите в него код:

```
namespace TaskApi.Models;

public class UpdateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public bool IsCompleted { get; set; }
    public string Priority { get; set; } = "Normal";
}
```

Практическое задание

1. Убедитесь, что проект компилируется без ошибок:

cd TaskApi

dotnet build

2. Сделайте скриншот терминала с успешной сборкой
3. Сохраните в папку `img` с именем: **step2_modelsLab29_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 2: TaskItem model and DTOs created"

git push

cd TaskApi

Шаг 3. Первый контроллер — хранилище в памяти

3.1. Что такое контроллер?

Контроллер — это класс, который содержит методы-обработчики для группы связанных маршрутов. Все маршруты одного контроллера объединяются под общим URL-префиксом.

TasksController → обрабатывает все запросы на `/api/tasks`

UsersController → обрабатывает все запросы на `/api/users`

3.2. Создание контроллера

В ЛР №28 мы хранили данные в отдельном классе **GamesStore**. Здесь данные лежат внутри самого контроллера — это проще, но менее гибко. В реальных проектах данные хранятся в базе данных.

Создайте папку **Controllers** и в ней файл **TasksController.cs**. Вручную перепишите код:

```
using Microsoft.AspNetCore.Mvc;
using TaskApi.Models;
namespace TaskApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class TasksController : ControllerBase { }
```

Проверьте что класс называется **TasksController** (буква **S**, частая ошибка)

Далее внутри класса **ControllerBase** создаём временное хранилище в памяти (вместо базы данных). **static** — список живёт пока работает сервер:


```

private static List<TaskItem> _tasks = new()
{
    new TaskItem {
        Id = 1,
        Title = "Изучить ASP.NET Core",
        Priority = "High",
        IsCompleted = true
    },
    new TaskItem {
        Id = 2,
        Title = "Сделать лабораторную №28",
        Priority = "High",
        IsCompleted = false
    },
    new TaskItem {
        Id = 3,
        Title = "Написать README",
        Priority = "Normal",
        IsCompleted = false
    },
};
// счётчик для генерации Id
private static int _nextId = 4;
}

```

Разберём атрибуты:

Атрибут	Что делает
[ApiController]	Включает удобные функции: автовалидацию, автоматический 400 при ошибках модели
[Route("api/[controller]")]	Задаёт базовый URL. [controller] заменяется на имя класса без слова Controller → /api/tasks
: ControllerBase	Базовый класс, который даёт методы Ok(), NotFound(), BadRequest() и др.

3.3. Метод GET — получить все задачи

Добавьте внутри класса **TasksController** следующий метод (перепишите вручную):

```

// GET /api/tasks
// GET /api/tasks?completed=true
[HttpGet]
public ActionResult<IEnumerable<TaskItem>> GetAll([FromQuery] bool?
completed = null) {
    var result = _tasks.AsEnumerable();

    // Фильтрация по статусу, если параметр передан
    if (completed.HasValue)
        result = result.Where(t => t.IsCompleted == completed.Value);

    return Ok(result);
}

```

Элемент	Описание
[HttpGet]	Этот метод обрабатывает GET-запросы
ActionResult<T>	Возвращаемый тип: либо данные типа T, либо HTTP-статус
[FromQuery]	Параметр берётся из строки запроса: ?completed=true
bool?	Nullable bool — параметр необязательный (может быть null)
Ok(result)	Возвращает HTTP 200 с данными
IEnumerable<T>	Интерфейс для «чего угодно, по чему можно пройтись».
	Принимает List, массив и т.д. Используем его вместо List<T>, чтобы не привязываться к конкретному типу коллекции
.AsEnumerable()	Превращает List в IEnumerable — нужно, чтобы потом можно было применить .Where() для фильтрации

3.4. Метод GET — получить одну задачу

Добавьте следующий метод:

```
// GET /api/tasks/1
[HttpGet("{id}")]
public ActionResult<TaskItem> GetById(int id) {
    var task = _tasks.FirstOrDefault(t => t.Id == id);
    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });
    return Ok(task);
}
```

task is null — современный синтаксис C# для проверки на null. Аналог `task == null` в JavaScript.

3.5. Метод POST — создать задачу

```
// POST /api/tasks
[HttpPost]
public ActionResult<TaskItem> Create([FromBody] CreateTaskDto dto) {
    if (string.IsNullOrWhiteSpace(dto.Title))
        return BadRequest(new { Message = "Поле Title обязательно для заполнения" });
    var newTask = new TaskItem {
        Id = _nextId++,
        Title = dto.Title,
        Description = dto.Description,
        Priority = dto.Priority,
        IsCompleted = false,
        CreatedAt = DateTime.Now
    };
    _tasks.Add(newTask);
    return CreatedAtAction(nameof(GetById), new { id = newTask.Id }, newTask);
}
```

string.IsNullOrWhiteSpace(dto.Title) — проверяет: строка пустая (""), состоит только из пробелов (" ") или вообще null? Это надёжнее, чем просто `dto.Title == ""`.

`_nextId++` — возвращает текущее значение и сразу увеличивает на 1. После этой строки `_nextId` уже больше на единицу — следующей задаче достанется следующий Id.

Элемент	Описание
[FromBody]	Данные берутся из тела запроса (JSON)
BadRequest(...)	Возвращает HTTP 400
CreatedAtAction(...)	Возвращает HTTP 201 + заголовок Location: /api/tasks/4
nameof(GetById)	Безопасная ссылка на имя метода (не строка, а выражение)

3.6. Метод PUT — обновить задачу

```
// PUT /api/tasks/1
[HttpPut("{id}")]
public ActionResult<TaskItem> Update(int id, [FromBody] UpdateTaskDto dto) {
    var task = _tasks.FirstOrDefault(t => t.Id == id);

    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });

    if (string.IsNullOrEmpty(dto.Title))
        return BadRequest(new { Message = "Поле Title не может быть пустым" });

    // Обновляем поля
    task.Title = dto.Title;
    task.Description = dto.Description;
    task.IsCompleted = dto.IsCompleted;
    task.Priority = dto.Priority;

    return Ok(task);
}
```

Обратите внимание: мы не пишем `task = dto`. Почему — разбирали в ЛР №28: `task` — это ссылка на объект в списке, и заменить её локально означает не изменить список

3.7. Метод DELETE — удалить задачу

```
// DELETE /api/tasks/1
[HttpDelete("{id}")]
public ActionResult Delete(int id) {
    var task = _tasks.FirstOrDefault(t => t.Id == id);

    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });

    _tasks.Remove(task);

    return NoContent(); // 204 — успешно, но возвращать нечего
}
```

3.8. Дополнительный метод PATCH — отметить выполненной

Иногда нужно обновить только одно поле, не передавая весь объект. Для этого используют PATCH:

```
// PATCH /api/tasks/1/complete
[HttpPatch("{id}/complete")]
public ActionResult<TaskItem> MarkComplete(int id) {
    var task = _tasks.FirstOrDefault(t => t.Id == id);

    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });

    task.IsCompleted = !task.IsCompleted; // toggle: true ↔ false

    return Ok(task);
}
```

PUT vs PATCH: PUT заменяет объект целиком — вы обязаны передать все поля. PATCH обновляет частично — только то, что нужно изменить. В нашем случае PATCH удобен для «отметить задачу выполненной», не передавая заново **title**, **description** и **priority**.

task.IsCompleted = !task.IsCompleted — оператор **!** инвертирует булево значение. Если было **true** → станет **false**, и наоборот. Это называется **toggle** (переключатель).

Практическое задание

1. Убедитесь, что проект компилируется:

dotnet build

2. Запустите сервер:

dotnet run

3. Откройте Swagger UI (**<http://localhost:XXXX/swagger>**) и убедитесь, что все 6 маршрутов появились в документации
4. Сделайте скриншот Swagger UI со списком маршрутов
5. Сохраните в папку **img** с именем: **step3_swaggerLab29_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 3: TasksController with full CRUD"

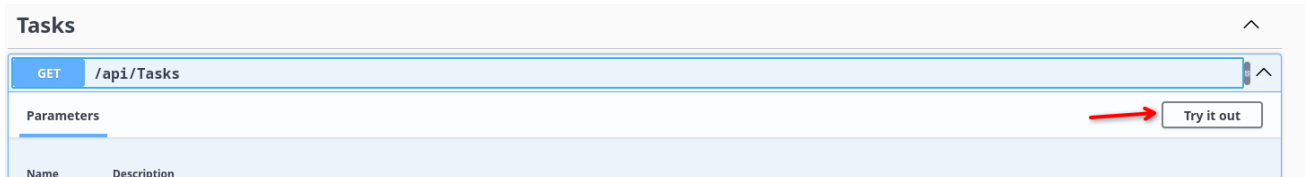
git push

cd TaskApi

Шаг 4. Тестирование через Swagger UI

4.1. GET /api/tasks — получить все задачи

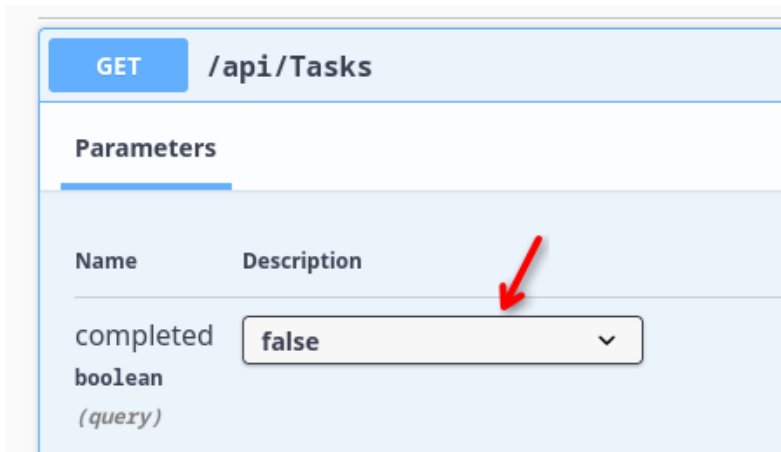
1. В Swagger UI нажмите на **GET /api/tasks**
2. Нажмите **Try it out**



3. Нажмите **Execute**
4. Убедитесь, что в разделе **Response body** отображается список из трёх задач, а **Code** равен 200

4.2. GET /api/tasks с фильтрацией

1. В том же методе в поле **completed** введите **false**



2. Нажмите **Execute**
3. Убедитесь, что вернулись только невыполненные задачи (2 штуки)

4.3. POST /api/tasks — создание задачи

1. Нажмите на **POST /api/tasks**
2. **Try it out** → в поле Request body замените содержимое на:

```
{  
  "title": "Тестовая задача от Swagger",  
  "description": "Создана через Swagger UI",  
  "priority": "Low"  
}
```

3. Нажмите **Execute**
4. Убедитесь, что код ответа — 201, а в теле ответа есть "id": 4

4.4. Проверка созданной задачи

1. Нажмите **GET /api/tasks/{id}**
2. В поле id введите 4
3. Нажмите **Execute**
4. Убедитесь, что вернулась ваша созданная задача

4.5. PUT /api/tasks — обновление задачи

1. Нажмите **PUT /api/tasks/{id}**
2. **id = 4**, тело запроса:

```
{  
  "title": "Обновлённая задача",  
  "description": "Описание изменено",  
  "isCompleted": true,  
  "priority": "High"  
}
```

3. Убедитесь, что вернулась задача с обновлёнными полями и кодом 200

4.6. Проверка несуществующего ресурса

1. Попробуйте **GET /api/tasks/{id}** с **id = 999**
2. Убедитесь, что вернулся код 404 и сообщение об ошибке

4.7. DELETE /api/tasks — удаление задачи

1. Нажмите **DELETE /api/tasks/{id}**
2. **id = 4**
3. Нажмите **Execute**
4. Убедитесь, что вернулся код 204 (без тела ответа)
5. Повторно выполните **GET /api/tasks/{id}** с **id = 4** — должен вернуть 404

Практическое задание

1. Сделайте скриншоты результатов нескольких запросов (минимум: POST 201, GET 200, DELETE 204, GET 404)
2. Сохраните в папку img с именами: **step4_swaggerTestsLab29_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 4: API tested via Swagger UI"

git push

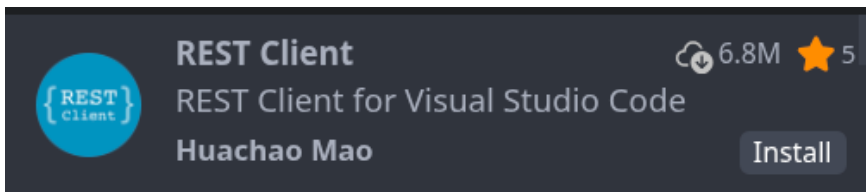
cd TaskApi

Шаг 5. Тестирование через REST Client в VS Code

Swagger удобен для быстрой проверки, но профессиональные разработчики хранят запросы в файлах **.http** прямо в репозитории. Это позволяет повторно использовать запросы и документировать их.

5.1. Установка расширения REST Client

1. Откройте Extensions (Ctrl+Shift+X)
2. Найдите **REST Client** (от Huachao Mao)



3. Установите расширение

5.2. Определение порта

Посмотрите в файле **Properties/launchSettings.json**, на каком порту запускается приложение:

```
"applicationUrl": "http://localhost:5101",
```

Запомните или запишите этот порт.

5.3. Создание файла с запросами

Создайте файл **requests.http** в корне папки **TaskApi** и вручную перепишите все запросы:

```
### Переменная — базовый URL (замените порт если отличается)  
10 references  
@baseUrl = http://localhost:5101
```

⚠ Порт в @baseUrl — замените на свой из launchSettings.json!

```
### GET  
### Получить все задачи  
Send Request  
GET {{baseUrl}}/api/tasks  
  
### Получить только выполненные задачи  
Send Request  
GET {{baseUrl}}/api/tasks?completed=true
```

ОБРАТИТЕ ВНИМАНИЕ В .http файлах каждый запрос должен быть отделён от следующего через ###, а между заголовками и телом запроса

обязательна пустая строка — иначе REST Client воспринимает { как имя заголовка.

```
### Получить только невыполненные задачи
Send Request
GET {{baseUrl}}/api/tasks?completed=false

### Получить задачу по id
Send Request
GET {{baseUrl}}/api/tasks/1

### Получить несуществующую задачу (ожидаем 404)
Send Request
GET {{baseUrl}}/api/tasks/999

### POST
### Создать новую задачу
Send Request
POST {{baseUrl}}/api/tasks
Content-Type: application/json

{
  "title": "Задача из REST Client",
  "description": "Создана через файл .http",
  "priority": "High"
}

### Попытка создать задачу без заголовка (ожидаем 400)
Send Request
POST {{baseUrl}}/api/tasks
Content-Type: application/json

{
  "title": "",
  "description": "Описание без заголовка",
  "priority": "Normal"
}

### PATCH
### Переключить статус задачи с id=2
Send Request
PATCH {{baseUrl}}/api/tasks/2/complete
```



```
### PUT
### Обновить задачу с id=1
Send Request
PUT {{baseUrl}}/api/tasks/1
Content-Type: application/json

{
  "title": "Изучить ASP.NET Core (обновлено)",
  "description": "Контроллеры, маршруты, DTO",
  "isCompleted": true,
  "priority": "High"
}
```

```
### DELETE
### Удалить задачу (сначала создайте новую через POST)
Send Request
DELETE {{baseUrl}}/api/tasks/4
```

5.4. Выполнение запросов

1. Запустите сервер (**dotnet run**)
2. Откройте файл **requests.http** в VS Code
3. Над каждым запросом появится надпись **Send Request** — кликайте по ней
4. Ответ откроется в новой панели справа

5.5. Проверка всех запросов

Выполните все запросы по порядку и убедитесь в правильных кодах ответа:

Запрос	Ожидаемый код
GET все задачи	200
GET по id (существующий)	200
GET по id (несуществующий)	404
POST с данными	201
POST без заголовка	400
PUT обновление	200
PATCH toggle	200
DELETE	204

Практическое задание

1. Сделайте скриншоты нескольких успешных запросов из REST Client (минимум 4 разных метода)
2. Сохраните в папку img с именем: **step5_restClientLab29_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 5: requests.http file for REST Client"

git push

cd TaskApi

Шаг 6. Расширение контроллера: поиск, сортировка и статистика

6.1. Добавляем маршрут поиска

Добавьте в **TasksController** новые методы (перепишите вручную):

```
// GET /api/tasks/search?query=ASP
[HttpGet("search")]
public ActionResult<IEnumerable<TaskItem>> Search([FromQuery] string query) {
    if (string.IsNullOrEmpty(query))
        return BadRequest(new { Message = "Параметр query не может быть пустым" });

    var results = _tasks
        .Where(t => t.Title.Contains(query, StringComparison.OrdinalIgnoreCase)
            || t.Description.Contains(query, StringComparison.OrdinalIgnoreCase))
        .ToList();

    return Ok(results);
}
```

StringComparison.OrdinalIgnoreCase — поиск без учёта регистра.

Аналог `.toLowerCase().includes(query)` в JavaScript.

6.2. Добавляем маршрут фильтрации по приоритету

```
// GET /api/tasks/priority/High
[HttpGet("priority/{level}")]
public ActionResult<IEnumerable<TaskItem>> GetByPriority(string level) {
    var allowed = new[] { "Low", "Normal", "High" };

    if (!allowed.Contains(level, StringComparer.OrdinalIgnoreCase))
        return BadRequest(new { Message = "Допустимые значения: Low, Normal, High" });

    var results = _tasks
        .Where(t => t.Priority.Equals(level, StringComparison.OrdinalIgnoreCase))
        .ToList();

    return Ok(results);
}
```

Маршруты с именованным сегментом (search, stats, sorted, priority/{level}) ASP.NET Core разрешает раньше, чем {id}, потому что буквенный путь точнее, чем параметр. Если бы level был int — возник бы конфликт

6.3. Добавляем маршрут статистики

```
// GET /api/tasks/stats
[HttpGet("stats")]
public ActionResult GetStats() {
    var total = _tasks.Count();
    var completed = _tasks.Count(t => t.IsCompleted);
    var pending = total - completed;

    var stats = new {
        Total = total,
        Completed = completed,
        Pending = pending,
        CompletionPct = total > 0 ? Math.Round((double)completed / total * 100, 1) : 0,
        ByPriority = new {
            High = _tasks.Count(t => t.Priority == "High"),
            Normal = _tasks.Count(t => t.Priority == "Normal"),
            Low = _tasks.Count(t => t.Priority == "Low"),
        }
    };

    return Ok(stats);
}
```

6.4. Добавляем маршрут сортировки

```
// GET /api/tasks/sorted?by=priority&desc=true
[HttpGet("sorted")]
public ActionResult<IEnumerable<TaskItem>> GetSorted(
    [FromQuery] string by = "id",
    [FromQuery] bool desc = false) {
    IEnumerable<TaskItem> sorted = by.ToLower() switch {
        "title" => _tasks.OrderBy(t => t.Title),
        "priority" => _tasks.OrderBy(t => t.Priority),
        "createdat" => _tasks.OrderBy(t => t.CreatedAt),
        _ => _tasks.OrderBy(t => t.Id), // по умолчанию — по id
    };

    if (desc)
        sorted = sorted.Reverse();

    return Ok(sorted);
}
```

switch выражение — современный синтаксис C#. Возвращает значение в зависимости от условия. Аналог цепочки if-else или объекта с ключами в JavaScript.

Практическое задание

1. Запустите **dotnet run** и в **Swagger** проверьте новые маршруты

2. Протестируйте через **REST Client** — добавьте в requests.http запросы для каждого нового маршрута:

```
### PATCH
### Переключить статус задачи с id=2
Send Request
PATCH {{baseUrl}}/api/tasks/2/complete

### DELETE
### Удалить задачу (сначала создайте новую через POST)
Send Request
DELETE {{baseUrl}}/api/tasks/4

### Поиск задач
Send Request
GET {{baseUrl}}/api/tasks/search?query=ASP

### Задачи с высоким приоритетом
Send Request
GET {{baseUrl}}/api/tasks/priority/High

### Статистика
Send Request
GET {{baseUrl}}/api/tasks/stats

### Отсортированные задачи по приоритету
Send Request
GET {{baseUrl}}/api/tasks/sorted?by=priority&desc=false
```

3. Убедитесь, что все запросы работают корректно

4. Сделайте скриншоты результатов маршрута **/stats** и одного из поисковых запросов

5. Сохраните в папку **img** с именем: **step6_extraRoutesLab29_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 6: search, filter, stats and sort endpoints"

git push

cd TaskApi

Шаг 7. Изучение Program.cs — настройка Web API

7.1. Добавление CORS

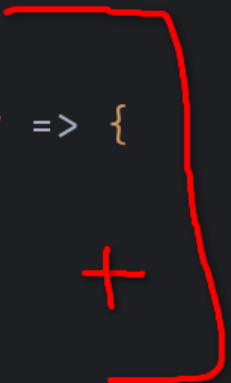
Если наш API будет использоваться фронтендом на другом порту (например, HTML-страница на file:// или localhost:3000), браузер заблокирует запросы из-за политики CORS.

Добавьте в **Program.cs** настройку CORS (вручную перепишите изменения):

```
app.UseHttpsRedirection();
app.UseCors("AllowAll"); // Включаем CORS
app.UseAuthorization();
```

Также добавим в разделе регистрации сервисов:

```
// Регистрация сервисов
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
// Добавляем CORS
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.AllowAnyOrigin()
               .AllowAnyMethod()
               .AllowAnyHeader();
    });
});
```



Почему это важно? В будущей лабораторной вы будете подключать фронтенд к вашему API. Без CORS браузер не пропустит запросы с другого порта. Мы настроили это заранее.

⚠ AllowAnyOrigin() разрешает запросы с любого сайта. Это удобно для разработки, но в реальном проекте нужно указывать конкретный адрес фронтенда: `.WithOrigins("http://localhost:3000")`

Практическое задание

1. Сделайте скриншот файла **Program.cs** с добавленным CORS
2. Сохраните в папку img с именем: **step7_programLab29_FIO.png**

Выполните в терминале:

```
cd ..
```

git add .

git commit -m "Step 7: CORS configured in Program.cs"

git push

cd TaskApi

Самостоятельное задание

Цель задания: создать подробное описание лабораторной работы в файле README.md.

Что должно быть в README.md:

1. Заголовок

2. Основная информация:

****ФИО:**** Иванов Иван Иванович

****Группа:**** ИСП-231

****Дата:**** дд.мм.гггг

3. Краткое описание работы (что изучили)

4. Структура проекта (дерево файлов)

5. Список всех реализованных маршрутов в виде таблицы

6. Итоговая таблица ASP.NET Core Controller-based API

7. Главные выводы (минимум 4)

Итоговая таблица (обязательно включить в README.md):

Аспект	ASP.NET Core Controllers
Маршруты	[HttpGet] атрибут над методом
Группировка маршрутов	Класс-контроллер
Базовый URL	[Route("api/[controller]")]
Параметр пути	(int id) — параметр метода
Параметр запроса	[FromQuery] bool? completed
Тело запроса	[FromBody] CreateTaskDto dto
Ответ 200	return Ok(data)
Ответ 201	return CreatedAtAction(...)
Ответ 404	return NotFound(...)
Ответ 204	return NoContent()
Типизация данных	Строгая (C#)
Документация	Swagger — устанавливается отдельно

Главные выводы:

1. REST — не протокол, а архитектурный стиль. Те же принципы работают с любым языком и фреймворком.

2. Контроллер в ASP.NET Core = Router в Express, только с автоматической документацией и строгой типизацией.
3. DTO защищает API от некорректных данных: клиент передаёт только то, что сервер разрешает принять.
4. Swagger UI позволяет тестировать API без Postman и без написания тестового JavaScript-кода.
5. Правильные HTTP-статусы — часть «контракта» API. Клиент должен понимать, что произошло, по коду ответа.

После выполните в терминале:

cd ..

git add .

git commit -m "Added complete README for Lab29"

git push

cd TaskApi

Итоговая таблица: что изучили в лабораторной

Концепция / команда	Описание
dotnet new webapi	Создать проект Web API с контроллерами и Swagger
[ApiController]	Атрибут, включающий автоматическую валидацию и обработку ошибок
[Route("api/[controller]")]	Базовый URL для всех маршрутов контроллера
[HttpGet], [HttpPost] и т.д.	Атрибуты, задающие HTTP-метод маршрута
[FromBody]	Данные берутся из тела JSON-запроса
[FromQuery]	Данные берутся из строки запроса (?key=value)
ActionResult<T>	Тип возврата: данные или HTTP-статус
Ok(data)	HTTP 200 с данными
CreatedAtAction(...)	HTTP 201 с заголовком Location
NotFound(...)	HTTP 404
BadRequest(...)	HTTP 400
NoContent()	HTTP 204
DTO	Отдельный класс для входящих данных (без служебных полей)
Swagger UI	Автодокументация и тестирование API в браузере
REST Client (VS Code)	Тестирование API через файлы .http прямо в редакторе
CORS	Политика разрешений для запросов с других доменов/портов